

The background of the slide features a dark blue field with vertical columns of binary code (0s and 1s) in a light blue, monospace font. Interspersed among the code are numerous out-of-focus circular light spots in shades of blue and orange, creating a bokeh effect.

02

Introduction to Pandas



About Pandas

- Open-source Python library
- Provides high-performance data manipulation and analysis tools

≡ ChatGPT



How to import pandas?

To import pandas, visit China or a zoo! But be ready—pandas come with lots of bamboo cravings!



DataFrames in Pandas

- Key Feature of Pandas
- 2-dimensional structure
- Consists of 3 principal components: data, rows and columns
- Labelled axes (rows and columns)
- Can Perform Arithmetic operations on rows and columns

The diagram illustrates a DataFrame structure. A table with 7 rows and 6 columns is shown. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed from 0 to 6. A label 'Columns' with arrows points to the column headers. A label 'Rows' with arrows points to the row indices. A label 'Data' with a box highlights the data cells. The data is as follows:

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

OG



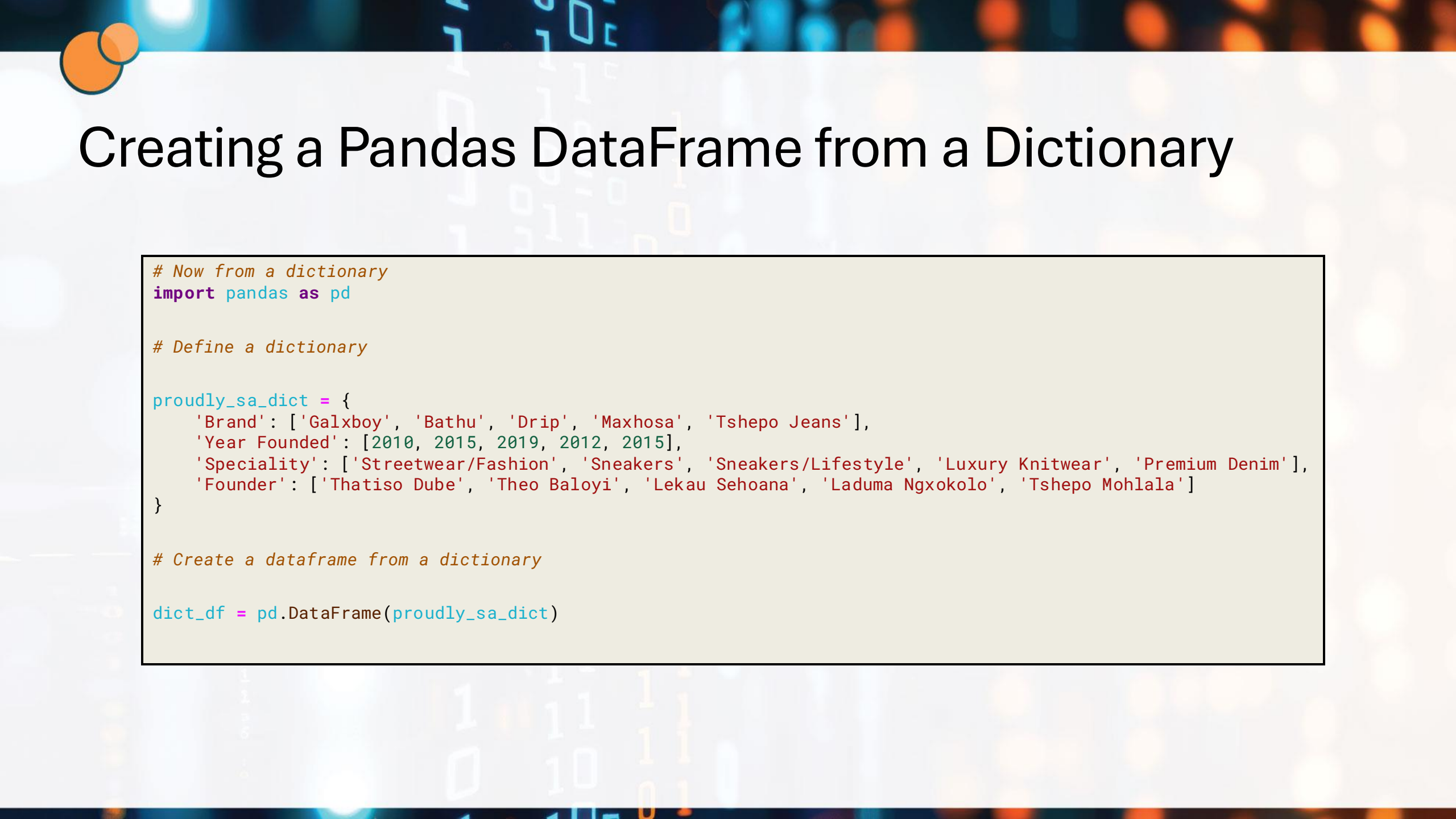
Creating a Pandas DataFrame

- First you need to import pandas | `import pandas as pd`
- Can be created from:
 - * List
 - * Dictionary
 - * List of Dictionaries



Creating a Pandas DataFrame from a List

```
# We first import pandas  
import pandas as pd  
  
# using a list  
  
proudly_sa = ['Bathu', 'Drip', 'K-Way', 'Maxhosa', 'Tshepo Jeans']  
  
# Create a DataFrame  
list_df = pd.DataFrame(proudly_sa)
```



Creating a Pandas DataFrame from a Dictionary

```
# Now from a dictionary
import pandas as pd

# Define a dictionary

proudly_sa_dict = {
    'Brand': ['Galxboy', 'Bathu', 'Drip', 'Maxhosa', 'Tshepo Jeans'],
    'Year Founded': [2010, 2015, 2019, 2012, 2015],
    'Speciality': ['Streetwear/Fashion', 'Sneakers', 'Sneakers/Lifestyle', 'Luxury Knitwear', 'Premium Denim'],
    'Founder': ['Thatiso Dube', 'Theo Baloyi', 'Lekau Sehoana', 'Laduma Ngxokolo', 'Tshepo Mohlala']
}

# Create a dataframe from a dictionary

dict_df = pd.DataFrame(proudly_sa_dict)
```



Querying a Pandas DataFrame

```
# Now from a dictionary
import pandas as pd

# Define a dictionary

proudly_sa_dict = {
    'Brand': ['Galxboy', 'Bathu', 'Drip', 'Maxhosa', 'Tshepo Jeans'],
    'Founded': [2010, 2015, 2019, 2012, 2015],
    'Speciality': ['Streetwear/Fashion', 'Sneakers', 'Sneakers/Lifestyle', 'Luxury Knitwear', 'Premium Denim'],
    'Founder': ['Thatiso Dube', 'Theo Baloyi', 'Lekau Sehoana', 'Laduma Ngxokolo', 'Tshepo Mohlala']
}

# Create a dataframe from a dictionary

dict_df = pd.DataFrame(proudly_sa_dict)

# querying a dataframe column

founders = dict_df['Founder']
```



Querying a Pandas DataFrame

```
# Querying a specific value

proudly_sa_dict = {
    'Brand': ['Galxboy', 'Bathu', 'Drip', 'Maxhosa', 'Tshepo Jeans'],
    'Founded': [2010, 2015, 2019, 2012, 2015],
    'Speciality': ['Streetwear/Fashion', 'Sneakers', 'Sneakers/Lifestyle', 'Luxury Knitwear', 'Premium Denim'],
    'Founder': ['Thatiso Dube', 'Theo Baloyi', 'Lekau Sehoana', 'Laduma Ngxokolo', 'Tshepo Mohlala']
}

# Create a dataframe from a dictionary
dict_df = pd.DataFrame(proudly_sa_dict)
# Querying owner of Drip
drip_owner = dict_df['Founder'][2]
```




DataFrames from External Sources

- In the real world, a Pandas DataFrame is usually created from external sources such as:



Databases



Excel Files



CSV Files



And More..

DataFrame From CSV

```
# imports
import pandas as pd

# specify the file path
my_file_path = 'drive/MyDrive/Colab Notebooks/hcd/data/sa_loan_eligibility_data.csv'

# Read the CSV file into a DataFrame
df = pd.read_csv(my_file_path)

df.head() # show the first 5 rows
df.tail() # last 5 rows
```





DataFrame From Excel

```
# imports
import pandas as pd

# specify the file path
my_file_path = 'drive/MyDrive/Colab Notebooks/hcd/data/sa_loan_eligibility_data.xlsx'

# Read the CSV file into a DataFrame
df = pd.read_excel(my_file_path)

df.head() # show the first 5 rows
df.tail() # last 5 rows
```





Dealing with Incomplete Data

Most real-world data is not perfect and sometimes:

1. Sheets don't have headers
2. There are more than one sheet in the Excel File
3. Information at the top of Excel sheet before table
4. Headers exists, but we would like to rename them to fit our narrative.



Excel/Data with no Headers

```
# You can specify to pandas that there are no headers:  
df_no_headers = pd.read_excel(no_headers_file, header=None)
```

```
# It then defaults to numbering the columns  
df_no_headers.head()
```

```
# You can then create your 'own' headers  
header_names = ['Full_Name', 'Province', 'Age', 'Income', ...']  
  
# specify your header names  
df_with_custom_headers = pd.read_excel(no_headers_file, header=None,  
names=header_names)  
  
df_with_custom_headers.head()
```




Excel with multiple sheets

```
# To get a specific sheet. You specify the sheet name  
df_soccer_stats = pd.read_excel(many_sheets_file, sheet_name='soccer_probability')  
df_soccer_stats.head()
```



Information at the top of Excel sheet before table

```
# To get the correct info, skip the rows that don't include the real data  
df_clean_excel = pd.read_excel(dirty_excel_file, skiprows=15)  
df_clean_excel.head()
```



Renaming Columns

```
# What if we wanted to rename 'Income' to 'Monthly Income'  
df = df.rename(columns={'Income': 'Monthly Income'})  
df.head()
```



Exporting DataFrames

After processing, data often needs to be exported for further use. We'll look into:

1. Exporting a DataFrame to an Excel File
2. Exporting a DataFrame to a CSV file



Exporting a DataFrame to a CSV file [df.to_csv()]

```
# Let us pretend the dictionary dataframe is from some process...

# first let us specify the file path
file_path = 'drive/MyDrive/Colab Notebooks/hcd/data/proudly_sa.csv'

# write to a csv
dict_df.to_csv(file_path)
```




Exporting a DataFrame to a CSV file [df.to_excel()]

```
# Let us pretend the dictionary dataframe is from some process...

# first let us specify the file path
file_path = 'drive/MyDrive/Colab Notebooks/hcd/data/proudly_sa.xlsx'

# write to a excel
dict_df.to_excel(file_path)
```



Modifying DataFrames

In most instances, the data from a source is not perfect requires modifications.

1. Replacing and removing entries
2. Removing unwanted rows and columns

[Covered in Data Exploration in more detail]



Replacing all df entries

	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Win Probability
0	Dondol Stars	ABC Motsepe	Orbit College	Home	Rainy	--	0.53
1	Dondol Stars	ABC Motsepe	Mamelodi Sundowns	Home	Night Match	0	0.67
2	Orlando Pirates	PSL	Bush Bucks	Home	Sunny	1	0.59
3	Bush Bucks	ABC Motsepe	Platinum City Rovers	Home	Night Match	3	0.66
4	Richards Bay	PSL	Orlando Pirates	Away	Night Match	--	0.50

```
# import numpy for numerical operations
import numpy as np

# Let us replace -- on numbered columns with nan
df_clean = df_dirty_excel.replace('--', np.nan)

df_clean.head()
```

Replacing from a specific column

	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)	Shots on Target	Yellow Cards	Win Probability
0	Dondol Stars	ABC Motsepe	Santos FC	NaN	Night Match	1	32752.0	11.1	60.8	1.0	0.0	0.53
1	Mpheni Home Defenders	PSL	Orbit College	NaN	Derby	1	17255.0	34.4	48.9	0.0	4.0	0.50
2	Black Leopards	NFD	Venda FC	Away	Night Match	1	NaN	NaN	59.8	8.0	5.0	0.83
3	NaN	ABC Motsepe	Cape Town Spurs	Home	Derby	6	39233.0	17.8	44.3	7.0	NaN	0.34

```
# Replacing in specific columns.
```

```
df_clean['Team Name'] = df_clean['Team Name'].replace(np.nan, 'Unknown Team')  
df_clean['Location'] = df_clean['Location'].replace(np.nan, 'Unknown Location')
```

```
df_clean.head()
```

Removing unwanted rows

	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)	Shots on Target	Yellow Cards	Win Probability
0	Dondol Stars	ABC Motsepe	Santos FC	--	Night Match	1	32752	11.1	60.8	1	0	0.53
1	Mpheni Home Defenders	PSL	Orbit College	--	Derby	1	17255	34.4	48.9	0	4	0.5
2	Black Leopards	NFD	Venda FC	Away	Night Match	1	--	--	59.8	8	5	0.83
3	--	ABC Motsepe	Cape Town Spurs	Home	Derby	6	39233	17.8	44.3	7	--	0.34

```
# If we include the original dataframe, we will only get instances where the result is true
```

```
df_clean = df_dirty_excel[~df_dirty_excel['Team Name'].isin(['--'])]
```

```
df_clean
```

After removing the rows, the index needs to be reset

	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)	Shots on Target	Yellow Cards	Win Probability
0	Dondol Stars	ABC Motsepe	Santos FC	--	Night Match	1	32752	11.1	60.8	1	0	0.53
1	Mpheni Home Defenders	PSL	Orbit College	--	Derby	1	17255	34.4	48.9	0	4	0.5
2	Black Leopards	NFD	Venda FC	Away	Night Match	1	--	--	59.8	8	5	0.83
5	Bush Bucks	NFD	Uppington City	Home	Sunny	0	28727	--	40.8	3	3	0.62

```
# If we include the original dataframe, we will only get instances where the result is true
```

```
df_clean = df_clean.reset_index()
```

```
df_clean
```




Removing Columns | Using Drop

	index	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)
0	0	Dondol Stars	ABC Motsepe	Santos FC	--	Night Match	1	32752	11.1	60.8
1	1	Mpheni Home Defenders	PSL	Orbit College	--	Derby	1	17255	34.4	48.9
2	2	Black Leopards	NFD	Venda FC	Away	Night Match	1	--	--	59.8
3	5	Bush Bucks	NFD	Uppington City	Home	Sunny	0	28727	--	40.8

```
# We want to remove League and index  
# Using Drop
```

```
df = df_clean.drop(columns=['League', 'index'])
```

Removing Columns | Using Filter

	index	Team Name	League	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)
0	0	Dondol Stars	ABC Motsepe	Santos FC	--	Night Match	1	32752	11.1	60.8
1	1	Mpheni Home Defenders	PSL	Orbit College	--	Derby	1	17255	34.4	48.9
2	2	Black Leopards	NFD	Venda FC	Away	Night Match	1	--	--	59.8
3	5	Bush Bucks	NFD	Uppington City	Home	Sunny	0	28727	--	40.8

```
# We want to keep only Team Name, Opponent and Win Probability
# Using Filter

df = df_clean.filter(['Team Name', 'Opponent', 'Win Probability'], axis=1) # axis=1 for columns
```

Removing Duplicates

	Team Name	Competition	Opponent	Location	Match Condition	Goals Last 3	Attendance	Temperature (°C)	Team Possession (%)	Shots on Target	Yellow Cards	Win Probability
0	Dondol Stars	ABC Motsepe	Santos FC	Home	Night Match	1	32752	11.1	60.8	1	0	0.53
1	Dondol Stars	ABC Motsepe	Santos FC	Home	Night Match	1	32752	11.1	60.8	1	0	0.53
2	Mpheni Home Defenders	PSL	Orbit College	Home	Derby	1	17255	34.4	48.9	0	4	0.5
3	Mpheni Home Defenders	PSL	Orbit College	Home	Derby	1	17255	34.4	48.9	0	4	0.5
4	Black Leopards	NFD	Venda FC	Away	Night Match	1	--	--	59.8	8	5	0.83

Removing Duplicates

```
df_no_dups = df.drop_duplicates(subset=['Team Name', 'Opponent', 'Location', 'Win Probability'], keep='first')
```

```
# Have an option of completely dropping by setting keep = False
```

```
# You can keep the last entry by specifying keep = 'last'
```

```
df_no_dups.head()
```



Dealing with Dates

```
# Change the string date to type datetime
```

```
df['Match Date'] = pd.to_datetime(df['Match Date'], format="%Y-%m-%d")
```

```
# Let us combine all the separate columns of date into a single Match Date
```

```
df['Match Date'] = pd.to_datetime(df[['Year', 'Month', 'Day']], format="%Y-%m-%d",  
errors='coerce')
```

```
# coerce check for invalid dates
```



```
return 0;
```

